



VIT[®]
BHOPAL

MULTI-DISEASE PREDICTION SYSTEM

A Machine Learning Based Risk & Symptom Prediction Application

VITyarthi — Build Your Own Project

Project Report / Presentation

Student Name:- SAI KUMAR

Registration No.:-25MIB10027

Course / Subject:-Fundamentals in AI and ML

Problem Statement

- Most people have no easy way to check their risk of conditions like diabetes or heart disease outside a clinical setting.
- Everyday illnesses (cold, flu, COVID-19, migraine, food poisoning) share overlapping symptoms and are often self-diagnosed incorrectly.
- Goal: build a single system that validates input, predicts risk/illness, stores prediction history, and reports on both the data and the models.

Objectives & Target Users

Objectives

- Train and serve 3 independent classification models

-
- Validate every input before it reaches a model
 - Persist every prediction (CRUD) for later review
 - Turn history and model metrics into readable analytics

Target Users

- Students / instructors evaluating applied ML and software engineering practice
- Hobbyist learners exploring a runnable ML-backed console application

Functional Requirements

The system is organised into three major functional modules:

1. Data Management

- Generate and clean datasets (diabetes, heart, symptoms)
- Train/test split with feature scaling

2. Prediction Engine

- Diabetes & Heart Disease risk (Random Forest)
- Symptom → Illness checker (Decision Tree)

3. Records & Reporting

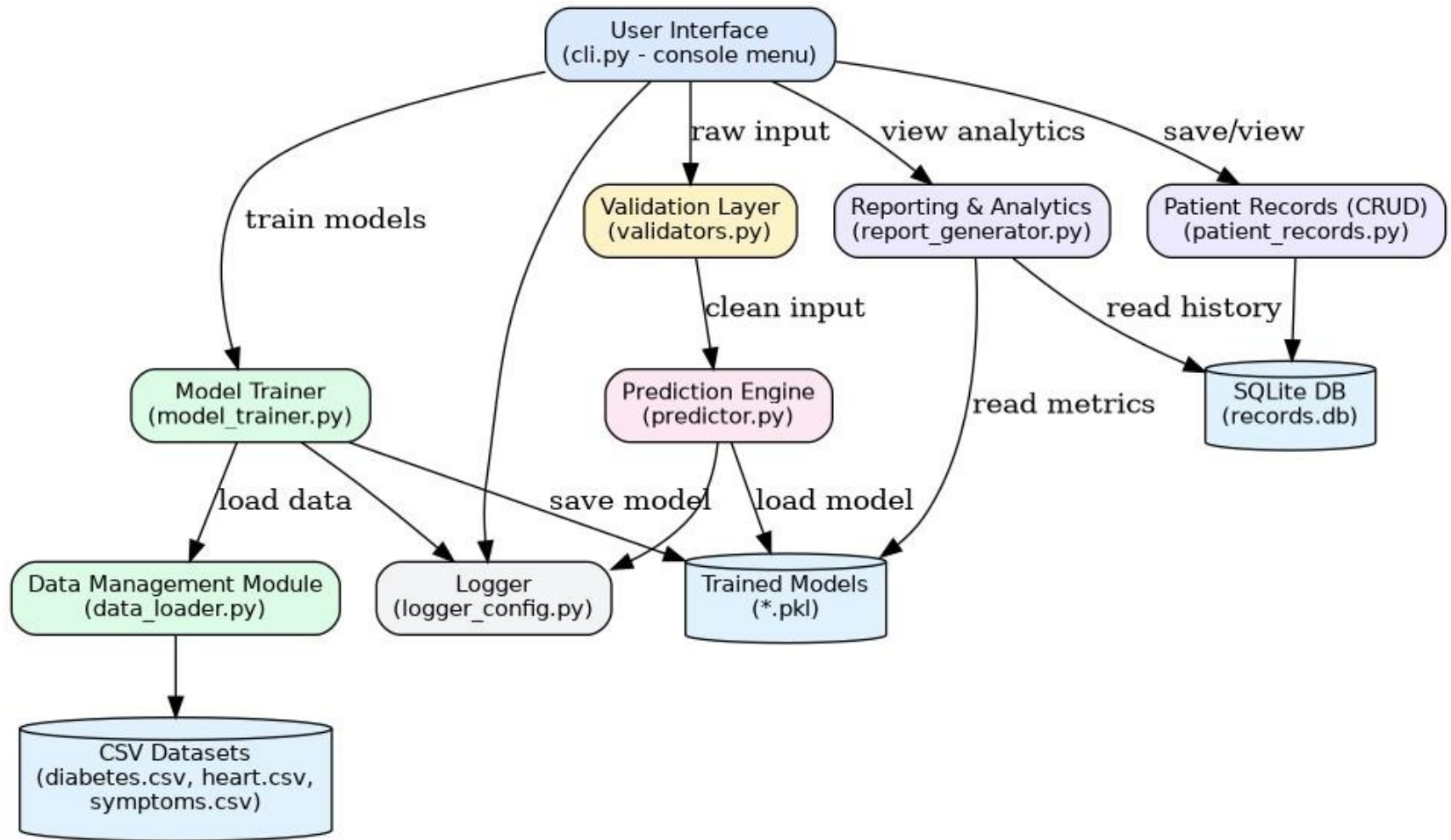
- CRUD patient history stored in SQLite
- Text summary + chart analytics

Non-Functional Requirements

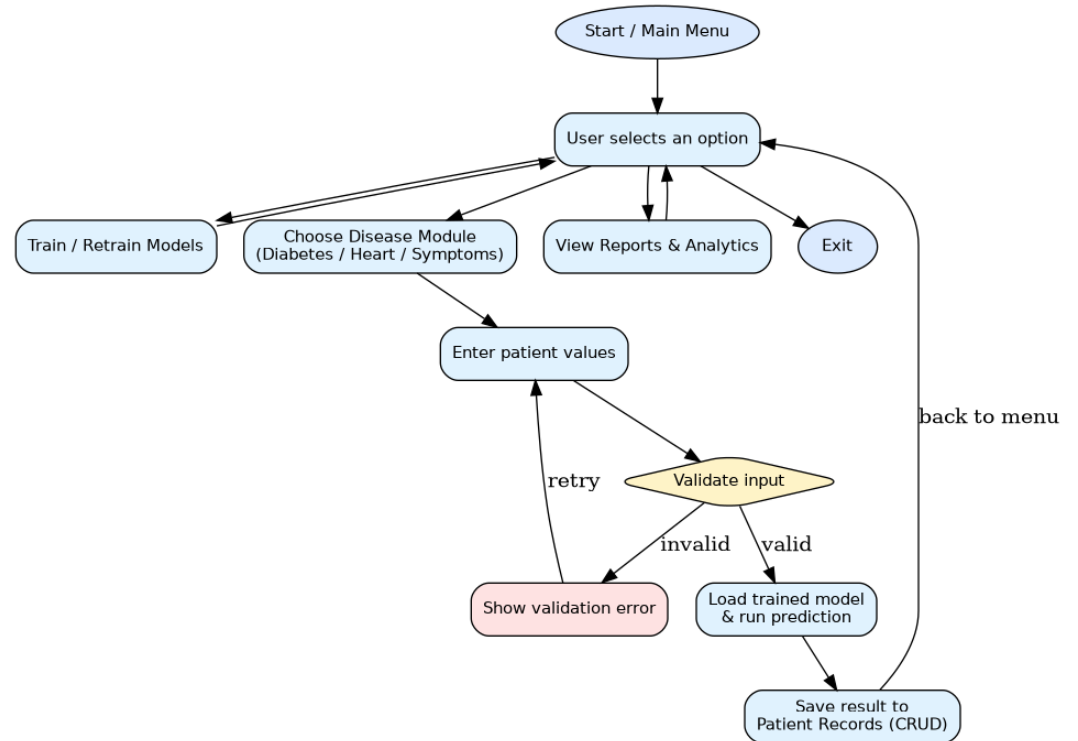
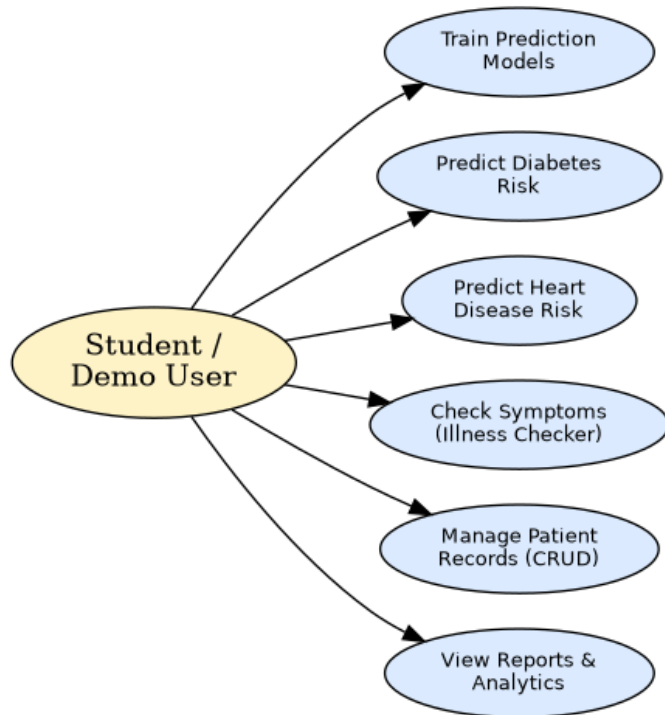
Performance	Sub-second training; near-instant predictions
Security	No network calls; all data stored locally

Reliability	try/except around every action; app never crashes on bad input
Scalability	Independent modules; dataset/model can be swapped easily
Maintainability	Each module has a single, documented responsibility
Logging & Error Handling	Rotating file logger + custom ValidationError class

System Architecture



Use Case & Process Flow Diagrams

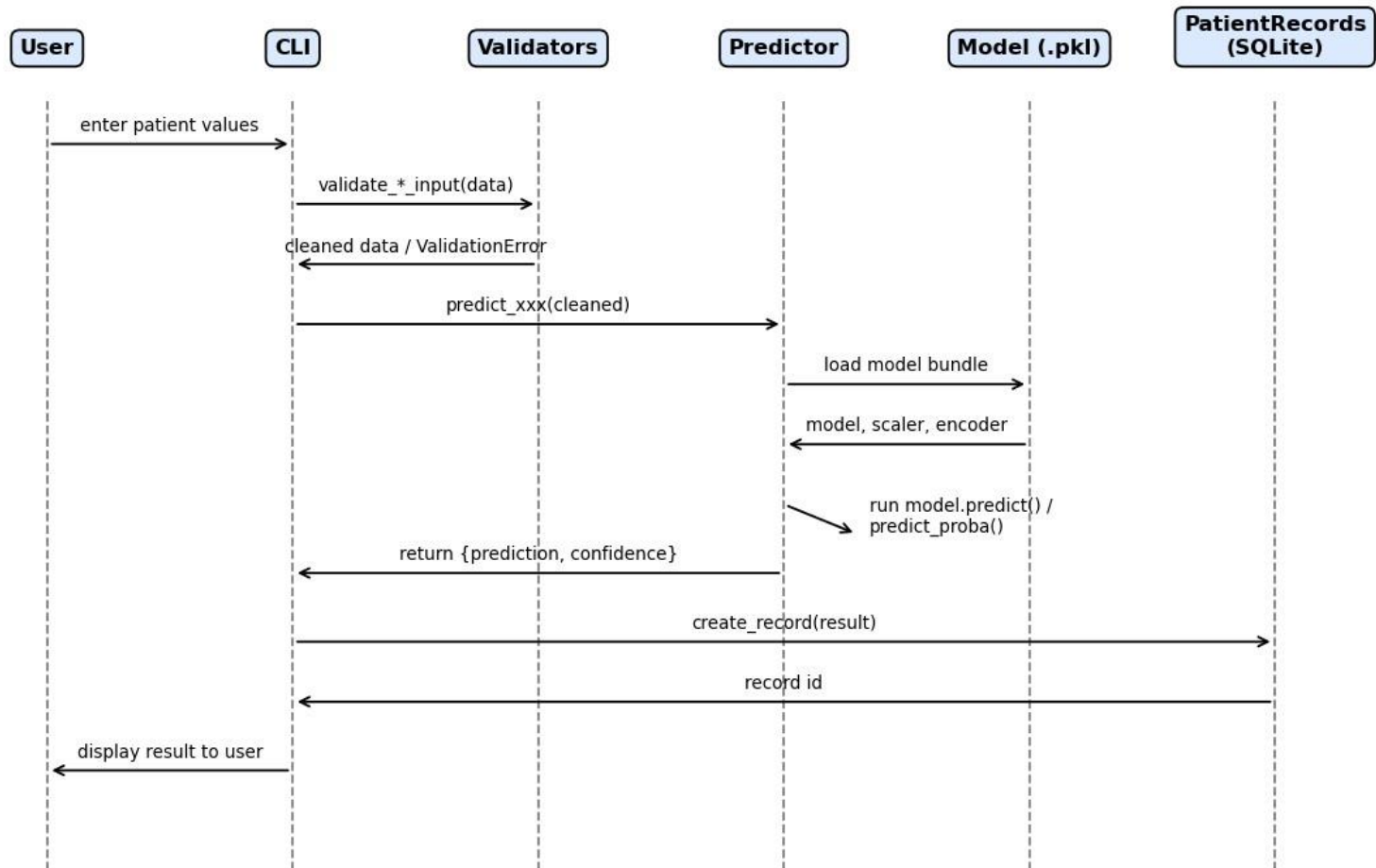


Use Case Diagram

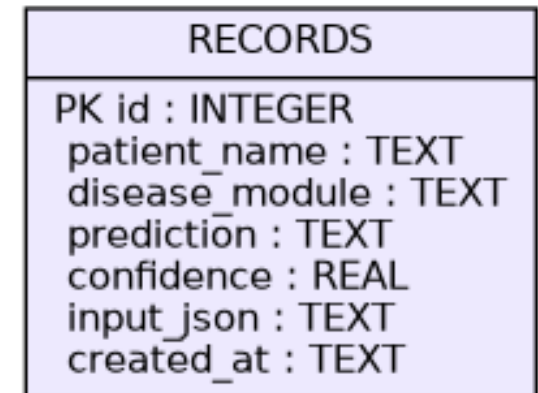
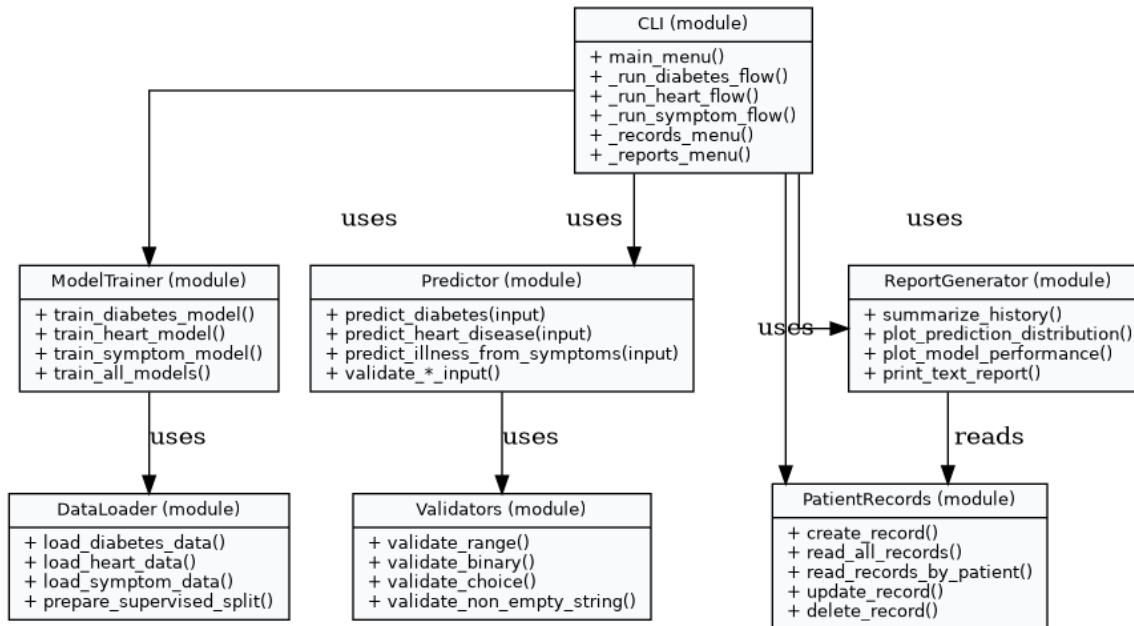
Process Flow Diagram

Sequence Diagram

Sequence Diagram — Running a Disease Prediction



Class Diagram & Database Design



Screenshots / Results

```
=====
MULTI-DISEASE PREDICTION SYSTEM (VITyarthi Project)
=====
Disclaimer: Educational demo only. Not medical advice.

=== MAIN MENU ===
1. Train / Retrain Prediction Models
2. Predict Diabetes Risk
3. Predict Heart Disease Risk
4. Symptom-Based Illness Checker
5. Patient Record Management (CRUD)
6. Reporting & Analytics
7. Exit
Choose an option: 2

--- Diabetes Risk Prediction ---
Patient name: Aarav Sharma
Pregnancies (0-20): 2
Glucose level mg/dL (40-260): 190
Blood pressure mmHg (20-140): 88
Skin thickness mm (0-100): 32
Insulin level mu U/mL (0-900): 250
BMI (10-70): 38.5
Diabetes pedigree function (0.0-3.0): 0.9
Age (1-120): 55

Result: Positive (confidence: 86.4%)
Saved as record #1.
```

ID	Patient	Module	Prediction	Conf%	Created At
5	Karan Mehta	Symptom Checker	Migraine	97.5	2026-09-14T04:20:11
4	Sneha Iyer	Heart Disease	Negative	92.75	2026-09-14T04:20:11
3	Rohan Gupta	Heart Disease	Positive	85.58	2026-09-14T04:20:11
2	Priya Nair	Diabetes	Negative	98.11	2026-09-14T04:20:11
1	Aarav Sharma	Diabetes	Positive	86.4	2026-09-14T04:20:11

```
=== PREDICTION HISTORY SUMMARY ===
Total predictions made : 5
```

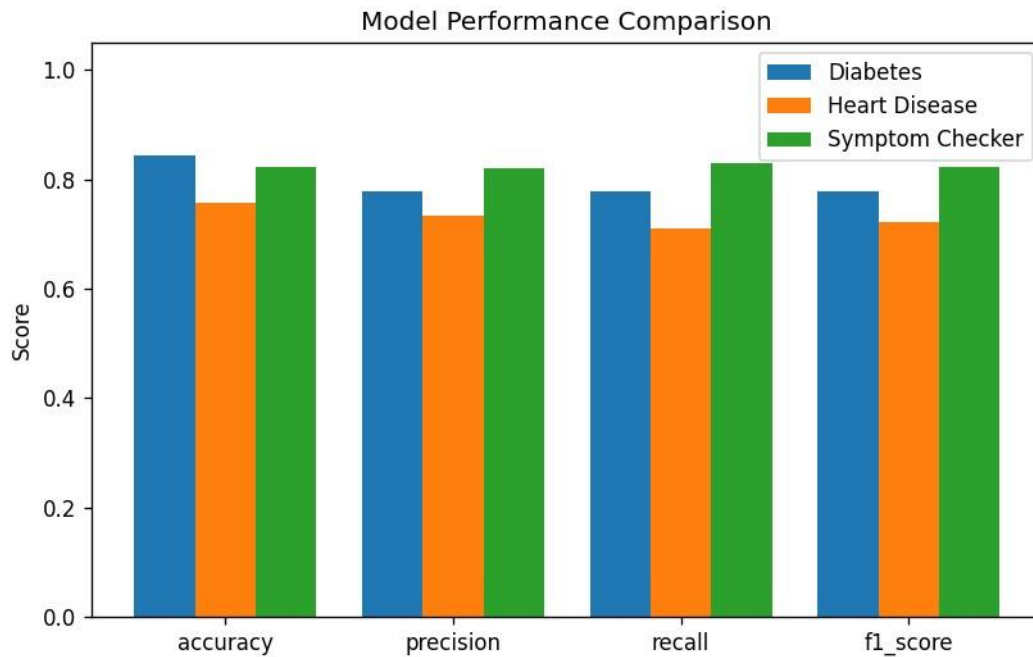
By module:

- Symptom Checker: 1
- Heart Disease: 2
- Diabetes: 2

By outcome:

- Symptom Checker - Migraine: 1
- Heart Disease - Negative: 1
- Heart Disease - Positive: 1
- Diabetes - Negative: 1
- Diabetes - Positive: 1

Model Performance



Model	Accuracy	F1
Diabetes (RF)	0.844	0.778
Heart Disease (RF)	0.757	0.721
Symptom Checker	0.822	0.823

Metrics measured on a held-out 20% test split.

Testing Approach

- Validator unit tests — range, binary, choice, and non-empty-string checks
- Data loader tests — dataset columns and row counts
- End-to-end model tests — trains all 3 models; asserts $\geq 60\%$ accuracy each
- Prediction tests — valid input accepted, out-of-range input rejected
- CRUD tests — isolated temp database: create, read, update, delete
- Result: 21 / 21 automated tests passing (pytest)